Title	BSL 11.1.1.d - Secure Coding Baseline - Details
Author(s)	Group Security - GRC
Issued by	Group Security
Catalogue no.	11.1.1
Date of issue	May 2023
Owner	Head of GRC
Last publication date	October 2025
Version	3
Classification	Internal
Document status	FINAL



Version	Date	Author	Comments
0.1	January 2023	Jolanda Kreijveld	Initial Draft
0.2	February 2023	Sita Chaudhury	Made changes/updates to initial draft.
0.3	March 2023	Michiel Stoop Sitakanta Chaudhury	Reviewed and updated Logical Access Control and Authentication and Password management controls Additional review on version 0.2
1.0	May 2023	Jeroen Mol	Final version
2.0	March 2024 April 2024	Prashant Shetty Jeroen Mol	Annual review: Updated the new template changes
3	October 2025	Prashant Shetty	Renamed BSL 11.1.1.d - Secure Coding Baseline Details to distinguish from new BSL 011.1.1; removed risks

Purpose & Scope
The Security in Development and Support Processes Policy, which is a child of the Enterprise Security Policy, and the Secure Development Standard describe the following objective(s) related to this theme specific Secure Coding Baseline:
* To ensure security is implemented as an integral part of all IT systems and to protect the services to associated risks throughout the complete development lifecycle. * To ensure that security is an integral part of the entire lifecycle of software, applications, and (mobile) apps development, from requirements definition until application decommissioning. * To prevent compromise, which may impact confidentiality, availability, and integrity.
This baseline is mandatory and defines the minimum set of security measures, also called security requirements, that are needed to meet these objectives.
The scope of this baseline are all IT systems, IT devices, IT services (e.g., operating systems (OS), middleware, databases, etc.), software, application and app development, maintenance, and support activities by Philips, on behalf of Philips, or related to Philips brands, including Philips enterprise solutions, throughout the complete development lifecycle. The standard applies to all employees, contingent workers, interns, contractors, third parties, consultants, and other personnel at Philips, including all personnel affiliated with third parties and working on Philips behalf.
Statement
The security controls mentioned in this baseline are supported by Security Management Framework policies and can be supported by additional standards and baselines defining the security requirements and controls that must be implemented and maintained to ensure that security and associated risks are managed properly.
Rules of Application
Use of the words **"must/shall"** indicate that compliance is mandatory. Use of the words **"should/may"** mean that compliance is normally expected and highly recommended, but dispensation may be allowed if there is a legitimate business case not to comply or appropriate compensatory controls are in place. Where an area of non-compliance with this policy is identified, it must be subject to a risk assessment. The risk assessment will consider the possible impact of a security breach resulting from the non-compliance. Based on the residual risk identified by the risk assessment, the non-compliance must be addressed, or a formal exception must be requested and approved.
Governance and Document Change Management
This baseline is owned, maintained, and issued by the Philips Group Security - GRC department in collaboration with the Secure Foundation Domain Lead Comments and change requests can be provided via the Security Management Framework's Improvement Request process, accessible on the Intranet:
SMF Improvement Request - New Item (philips.com)

Security Management Framework (SMF) Baseline Controls						Impact Area		
SMF Control ID	SMF Control Group	SMF Control Title	SMF Control Description	SMF Control / Parameter	SMF Control / Parameter Value	Confidentiality	Integrity	Availability
11.1.1_1	Secure Coding	Authentication & Password Management	Require authentication for all pages and resources, except those specifically intended to be public.			√	√	√
11.1.1_2	Secure Coding	Authentication & Password Management	All authentication controls must be enforced on a trusted system (e.g., The server/program/service) using the Philips approved and/or provided Identity and Access Management solutions. Use the Logical access control Standard as mandatory input.			√	√	√
11.1.1_3	Secure Coding	Authentication & Password Management	Establish and utilize standard, tested, authentication services whenever possible using the Philips approved and/or provided Identity and Access Management solutions. Use the Logical access control Standard as mandatory input			√	√	√
11.1.1_4	Secure Coding	Authentication & Password Management	Use a centralized implementation for all authentication controls, including libraries that call external authentication services using the Philips approved and/or provided Identity and Access Management solutions. Use the Logical access control Standard as mandatory input			√	√	√
11.1.1_5	Secure Coding	Authentication & Password Management	Segregate authentication logic from the resource being requested and use redirection to and from the centralized authentication control			√	√	√
11.1.1_6	Secure Coding	Authentication & Password Management	All authentication controls should fail securely			√	√	√
11.1.1_7	Secure Coding	Authentication & Password Management	All administrative and account management functions must be at least as secure as the primary authentication mechanism.			√	√	√
11.1.1_8	Secure Coding	Authentication & Password Management	If your application manages a credential store, it must ensure that only cryptographically strong one-way salted hashes of passwords are stored and that the table/file that stores the passwords and keys is write-able only by the application.	Do not use the MD5 algorithm if it can be avoided and never store passwords in plain text form in the database, use hashing algorithm to obfuscate such as the bcrypt.		√	√	√
11.1.1_9	Secure Coding	Authentication & Password Management	Password hashing must be implemented on a trusted system (e.g., The server) using the Philips approved and/or provided Identity and Access Management solutions. Use the Logical access control Standard as mandatory input			√	√	√
11.1.1_10	Secure Coding	Authentication & Password Management	Validate the authentication data only on completion of all data input, especially for sequential authentication implementations			√	√	√

11.1.1_11	Secure Coding	Authentication & Password Management	Authentication failure responses should not indicate which part of the authentication data was incorrect. For example, instead of "Invalid username" or "Invalid password", just use "Invalid username and/or password" for both. Error responses must be truly identical in both display and source code			√	√	√
11.1.1_12	Secure Coding	Authentication & Password Management	Utilize authentication for connections to external systems that involve sensitive information or functions			√	√	√
11.1.1_13	Secure Coding	Authentication & Password Management	Authentication credentials for accessing services external to the application should be encrypted and stored in a protected location on a trusted system. The source code is NOT a secure location. The Philips approved and/or provided Identity and Access Management solutions must be used. Use the Logical access control Standard as mandatory input			√	√	√
11.1.1_14	Secure Coding	Authentication & Password Management	Only send non-temporary passwords over an encrypted connection or as encrypted data, such as in an encrypted email.			√	√	√
11.1.1_15	Secure Coding	Authentication & Password Management	Enforce password and pincode complexity requirements established by the Philips Credential baseline, or regulation in case regulation requires higher requirement.			√	√	√
11.1.1_16	Secure Coding	Authentication & Password Management	Enforce password or pincode length requirements established by the Philips Credential baseline or regulation in case regulation requires higher password length.			√	√	√
11.1.1_17	Secure Coding	Authentication & Password Management	Password entry should be obscured on the user's screen.			√	√	√
11.1.1_18	Secure Coding	Authentication & Password Management	Enforce account disabling after an established number of invalid login attempts. The account must be disabled for a period of time sufficient to discourage brute force guessing of credentials, but not so long as to allow for a denial-of-service attack to be performed. The Philips Credential baseline must be followed for this.			√	√	√
11.1.1_19	Secure Coding	Authentication & Password Management	Password reset and changing operations require the same level of controls as account creation and authentication.			√	√	√
11.1.1_20	Secure Coding	Authentication & Password Management	Password reset must following the Philips rules on passwords as defined in the Philips Logical Access Control Policy, Standard and associated Baselines.			√	√	√
11.1.1_21	Secure Coding	Authentication & Password Management	If using email based resets, only send email to a pre-registered address with a temporary link/password.			√	√	√
11.1.1_22	Secure Coding	Authentication & Password Management	Temporary passwords and links should have an expiration time of max 20 minutes, according to the credential baseline.			√	√	√
11.1.1_23	Secure Coding	Authentication & Password Management	Password change must be enforced at first log on and adhere to the credential baseline.			√	√	√
11.1.1_24	Secure Coding	Authentication & Password Management	Notify users when a password reset occurs.			√	√	√
11.1.1_25	Secure Coding	Authentication & Password Management	Prevent password re-use according the credential baseline.			√	√	√
11.1.1_26	Secure Coding	Authentication & Password Management	Passwords should be at least one day old before they can be changed, to prevent attacks on password re-use.	A new password can be only be set/changed once per day as required in the credential baseline	1 day	√	√	√

11.1.1_27	Secure Coding	Authentication & Password Management	Enforce password changes based on requirements established in policy or regulation. Critical systems may require more frequent changes. The time between resets must be administratively controlled and adhere to the Credential Baseline.			√	√	√
11.1.1_28	Secure Coding	Authentication & Password Management	Change all vendor-supplied default passwords and user IDs or disable the associated accounts.	ID's for registered identities must never be re-used as required in the Identity and Account baseline.		√	√	√
11.1.1_29	secure Coding	Authentication & Password Management	Re-authenticate users prior to performing critical operations			√	√	√
11.1.1_30	Secure Coding	Authentication & Password Management	Use Multi-Factor Authentication for highly sensitive or high value transactional accounts as defined by the Philips Logical Access Control Policy, Standard and baselines.			√	√	√
11.1.1_31	Secure Coding	Authentication & Password Management	If using third party code for authentication, inspect the code carefully to ensure it is not affected by any malicious code			√	√	√
11.1.1_32	Secure Coding	Communications Security	Disable weak cipher suites and vulnerable versions of SSL/TLS	Use TLS 1.2 or greater and disable all null, export, 40-bit or DES cipher suites.		√	√	
11.1.1_33	Secure Coding	Communications Security	TLS certificates must be valid and have the correct domain name, not be expired, and be installed with intermediate certificates when required	Only TLS 1.2 or higher must be used and a valid certificate.		√	√	
11.1.1_34	Secure Coding	Communications Security	Failed TLS connections must not fall back to an insecure connection			√	√	
11.1.1_35	Secure Coding	Communications Security	Utilize TLS connections for all content requiring authenticated access and for all other sensitive information			√	√	
11.1.1_36	Secure Coding	Communications Security	Utilize TLS for connections to external systems that involve sensitive information or functions			√	√	
11.1.1_37	Secure Coding	Communications Security	Utilize a single standard TLS implementation that is configured appropriately.			√	√	
11.1.1_38	Secure Coding	Communications Security	Specify character encodings for all connections			√	√	
11.1.1_39	Secure Coding	Communications Security	Filter parameters containing sensitive information from the HTTP referer, when linking to external sites.			√	√	
11.1.1_40	Secure Coding	Cryptography	All cryptographic functions used to protect secrets from the application user must be implemented on a trusted system (e.g., The server) following the Philips Cryptography Security Standard and baseline in addition to the controls in this baseline.			√	√	
11.1.1_41	Secure Coding	Cryptography	Do not store any cryptographic keys in the code used for protecting secrets in your application code.	Keys must be stored in a vault.		√	√	
11.1.1_42	Secure Coding	Cryptography	Protect master secrets from unauthorized access (restricted to service access) and keep it at secure location (e.g., Vault). Nobody (even Admin) must be able to export the master key from the protected location.			√	√	
11.1.1_43	secure Coding	Cryptography	Cryptographic modules should fail securely			√	√	
11.1.1_44	secure Coding	Cryptography	All random numbers, random file names, random GUIDs, and random strings should be generated using the cryptographic module's approved random number generator when these random values are intended to be un-guessable			√	√	
11.1.1_45	secure Coding	Cryptography	Cryptographic modules used must comply to FIPS 140-2. See cryptography standard and baseline for more detailed information about FIPS 140-2			√	√	

11.1.1_46	secure Coding	Cryptography	The Philips Cryptography Standard and baseline must be complied with for key management.			√	√	
11.1.1_47	secure Coding	Data Protection	Implement least privilege, restrict users to only the functionality, data and system information that is required to perform their tasks. As defined in the Logical Access Control Standard.			√	√	√
11.1.1_48	secure Coding	Data Protection	Protect all cached or temporary copies of sensitive data stored on the server from unauthorized access and purge those temporary working files a soon as they are no longer required.			√	√	√
11.1.1_49	secure Coding	Data Protection	Encrypt highly sensitive stored information, like authentication verification data, even on the server side. Always use well vetted algorithms, see "Cryptographic" within this baseline and the Philips Cryptography Security standard and baseline for additional guidance.			√	√	√
11.1.1_50	secure Coding	Data Protection	Protect server-side source-code from being downloaded by a user			√	√	√
11.1.1_51	Secure Coding	Data Protection	Do not store passwords, connection strings or other sensitive information in clear text or in any non-cryptographically secure manner on the client side. This includes embedding in insecure formats like: MS viewstate, Adobe flash or compiled code.			√	√	√
11.1.1_52	Secure Coding	Data Protection	Remove comments in user accessible production code that may reveal backend system or other sensitive information			√	√	√
11.1.1_53	Secure Coding	Data Protection	Remove unnecessary application and system documentation as this can reveal useful information to attackers			√	√	√
11.1.1_54	Secure Coding	Data Protection	Do not include sensitive information in HTTP GET request parameters.			√	√	√
11.1.1_55	Secure Coding	Data Protection	Disable auto complete features on forms expected to contain sensitive information, including authentication			√	√	√
11.1.1_56	Secure Coding	Data Protection	Disable client side caching on pages containing sensitive information. Cache-Control: no-store, may be used in conjunction with the HTTP header control "Pragma: no-cache", which is less effective, but is HTTP/1.0 backward compatible.			√	√	√
11.1.1_57	Secure Coding	Data Protection	The application should support the removal of sensitive data when that data is no longer required. (e.g. personal information or certain financial data)			√	√	√
11.1.1_58	Secure Coding	Data Protection	Implement appropriate logical access controls for sensitive data stored on the server. This includes cached data, temporary files and data that should be accessible only by specific system users			√	√	√
11.1.1_59	Secure Coding	Data protection	Use industry accepted methods for the secure disposal of data from storage media and that data must not be recoverable by any forensic emans.			√	√	√
11.1.1_60	Secure Coding	Database Security	The Philips baseline for either generic or specific databases must be complied to if available, in addition to all secure coding controls mentioned in this secure coding standard, Website secure coding baseline and API Baseline.			√	√	√

11.1.1_61	Secure Coding	Database Security	Use strongly typed parameterized queries.	A parameterized query is a query in which placeholders are used for parameters and the parameter values are supplied at execution time.		√	√	√
11.1.1_62	Secure Coding	Database Security	Utilize input validation and output encoding of this secure coding baseline and be sure to address meta characters. If these fail, do not run the database command			√	√	√
11.1.1_63	Secure Coding	Database Security	Ensure that variables are strongly typed			√	√	√
11.1.1_64	Secure Coding	Database Security	The application should use the lowest possible level of privilege when accessing the database			√	√	√
11.1.1_65	Secure Coding	Database Security	Use secure credentials for database access			√	√	√
11.1.1_66	Secure Coding	Database Security	Connection strings should not be hard coded within the application. Connection strings should be stored in a separate configuration file on a trusted system and they should be encrypted.	Encryption must adhere to the Cryptography security standard and baseline.		√	√	√
11.1.1_67	Secure Coding	Database Security	Use stored procedures to abstract data access and allow for the removal of permissions to the base tables in the database.			√	√	√
11.1.1_68	Secure Coding	Database Security	Close the connection as soon as possible for the application.			√	√	√
11.1.1_69	Secure Coding	Database Security	Remove or change all default database administrative passwords as defined in the credential baseline.	Utilize strong passwords/phrases or implement multi-factor authentication		√	√	√
11.1.1_70	Secure Coding	Database Security	Turn off all unnecessary database functionality (e.g., unnecessary stored procedures or services, utility packages, install only the minimum set of features and options required (surface area reduction)).			√	√	√
11.1.1_71	Secure Coding	Database Security	Remove unnecessary default vendor content (e.g., sample schemas).			√	√	√
11.1.1_72	Secure Coding	Database Security	Disable any default accounts that are not required to support business requirements as defined in the identity & account baseline.			√	√	√
11.1.1_73	Secure Coding	Database Security	The application should connect to the database with different credentials for every trust distinction (e.g., user, read-only user, guest, administrators).			√	√	√
11.1.1_74	Secure Coding	Error Handling & Logging	Do not disclose sensitive information in error responses, including system details, session identifiers or account information.			√		
11.1.1_75	Secure Coding	Error Handling & Logging	Use error handlers that do not display debugging or stack trace information.			√		
11.1.1_76	Secure Coding	Error Handling & Logging	Implement generic error messages and use custom error pages.			√		
11.1.1_77	Secure Coding	Error Handling & Logging	The application should handle application errors and not rely on the server configuration.			√		
11.1.1_78	Secure Coding	Error Handling & Logging	Properly free allocated memory when error conditions occur.			√		
11.1.1_79	Secure Coding	Error Handling & Logging	Error handling logic associated with security controls should deny access by default.			√		
11.1.1_80	Secure Coding	Error Handling & Logging	All logging controls should be implemented on a trusted system (e.g., The server).			√		

11.1.1_81	Secure Coding	Error Handling & Logging	Logging controls should support both success and failure of specified security events.			√		
11.1.1_82	Secure Coding	Error Handling & Logging	Ensure logs contain important log event data.			√		
11.1.1_83	Secure Coding	Error Handling & Logging	Ensure log entries that include un-trusted data will not execute as code in the intended log viewing interface or software.			√		
11.1.1_84	Secure Coding	Error Handling & Logging	Restrict access to logs to only authorized individuals.			√		
11.1.1_85	Secure Coding	Error Handling & Logging	Utilize a master routine for all logging operations.			√		
11.1.1_86	Secure Coding	Error Handling & Logging	Do not store sensitive information in logs, including unnecessary system details, session identifiers or passwords.			√		
11.1.1_87	Secure Coding	Error Handling & Logging	Ensure that a mechanism exists to conduct log analysis.			√		
11.1.1_88	Secure Coding	Error Handling & Logging	Log all input validation failures.			√		
11.1.1_89	Secure Coding	Error Handling & Logging	Log all authentication attempts, especially failures.			√		
11.1.1_90	Secure Coding	Error Handling & Logging	Log all logical access control failures.			√		
11.1.1_91	Secure Coding	Error Handling & Logging	Log all apparent tampering events, including unexpected changes to state data.			√		
11.1.1_92	Secure Coding	Error Handling & Logging	Log attempts to connect with invalid or expired session tokens.			√		
11.1.1_93	Secure Coding	Error Handling & Logging	Log all system exceptions.			√		
11.1.1_94	Secure Coding	Error Handling & Logging	Log all administrative functions, including changes to the security configuration settings.			√		
11.1.1_95	Secure Coding	Error Handling & Logging	Log all backend TLS connection failures.			√		
11.1.1_96	Secure Coding	Error Handling & Logging	Log cryptographic module failures.			√		
11.1.1_97	Secure Coding	Error Handling & Logging	Use a cryptographic hash function to validate log entry integrity.			√		
11.1.1_98	Secure Coding	Error Handling & Logging	Logs (e.g. debug logs, system logs, etc.) should not contain any sensitive data.			√		
11.1.1_99	Secure Coding	File Management	Do not pass user supplied data directly to any dynamic include function.			√	√	√
11.1.1_100	Secure Coding	File Management	Require authentication before allowing a file to be uploaded.			√	√	√
11.1.1_101	Secure Coding	File Management	Limit the type of files that can be uploaded to only those types that are needed for business purposes.			√	√	√
11.1.1_102	Secure Coding	File Management	Validate uploaded files are the expected type by checking file headers. Checking for file type by extension alone is not sufficient.			√	√	√
11.1.1_103	Secure Coding	File Management	Do not save files in the same web context as the application. Files should either go to the content server or in the database.			√	√	√
11.1.1_104	Secure Coding	File Management	Prevent or restrict the uploading of any file that may be interpreted by the web server.			√	√	√
11.1.1_105	Secure Coding	File Management	Turn off execution privileges on file upload directories			√	√	√
11.1.1_106	Secure Coding	File Management	Implement safe uploading in UNIX by mounting the targeted file directory as a logical drive using the associated path or the chrooted environment.			√	√	√
11.1.1_107	Secure Coding	File Management	When referencing existing files, use a white list of allowed file names and types. Validate the value of the parameter being passed and if it does not match one of the expected values, either reject it or use a hard coded default file value for the content instead.			√	√	√

11.1.1_108	Secure Coding	File Management	Do not pass user supplied data into a dynamic redirect. If this must be allowed, then the redirect should accept only validated, relative path URLs.			√	√	√
11.1.1_109	Secure Coding	File Management	Do not pass directory or file paths, use index values mapped to pre-defined list of paths.			√	√	√
11.1.1_110	Secure Coding	File Management	Never send the absolute file path to the client.			√	√	√
11.1.1_111	Secure Coding	File Management	Ensure application files and resources are read-only.			√	√	√
11.1.1_112	Secure Coding	File Management	Scan user uploaded files for viruses and malware.			√	√	√
11.1.1_113	Secure Coding	General Coding Practices	Use tested and approved managed code rather than creating new unmanaged code for common tasks.			√	√	√
11.1.1_114	Secure Coding	General Coding Practices	Utilize task specific built-in APIs to conduct operating system tasks. Do not allow the application to issue commands directly to the Operating System, especially through the use of application initiated command shells.	The API Standard and baseline must be used and adhered to when coding/using APIs.		√	√	√
11.1.1_115	Secure Coding	General Coding Practices	Use checksums or hashes to verify the integrity of interpreted code, libraries, executables, and configuration files .			√	√	√
11.1.1_116	Secure Coding	General Coding Practices	Utilize locking to prevent multiple simultaneous requests or use a synchronization mechanism to prevent race conditions.			√	√	√
11.1.1_117	Secure Coding	General Coding Practices	Protect shared variables and resources from inappropriate concurrent access.			√	√	√
11.1.1_118	Secure Coding	General Coding Practices	Explicitly initialize all your variables and other data stores, either during declaration or just before the first usage.			√	√	√
11.1.1_119	Secure Coding	General Coding Practices	In cases where the application must run with elevated privileges, raise privileges as late as possible, and drop them as soon as possible.			√	√	√
11.1.1_120	Secure Coding	General Coding Practices	Avoid calculation errors by understanding your programming language's underlying representation and how it interacts with numeric calculation. Pay close attention to byte size discrepancies, precision, signed/unsigned distinctions, truncation, conversion and casting between types, "not-a-number" calculations, and how your language handles numbers that are too large or too small for its underlying representation.			√	√	√
11.1.1_121	Secure Coding	General Coding Practices	Do not pass user supplied data to any dynamic execution function.			√	√	√
11.1.1_122	Secure Coding	General Coding Practices	Restrict users from generating new code or altering existing code.			√	√	√
11.1.1_123	Secure Coding	General Coding Practices	Review all secondary applications, third party code and libraries to determine business necessity and validate safe functionality, as these can introduce new vulnerabilities			√	√	√
11.1.1_124	Secure Coding	General Coding Practices	Implement safe updating. If the application will utilize automatic updates, then use cryptographic signatures for your code and ensure your download clients verify those signatures. Use encrypted channels to transfer the code from the host server.			√	√	√

11.1.1_125	Secure Coding	Input Validation	Conduct all data validation on a trusted system (e.g., The server).	Systematic Check the accepts or reject data based on a set of rules : check for **Correctness:** e.g. quantity is numeric **Business rules**: e.g. reservation occures in the future **Warnings:** e.g. certain dates can not be selected because of holiday. **Information Security**: e.g Check on security threats like SQL commands **Risk Controls**: e.g. prevent orders beyond certain authorized amount			√	√
11.1.1_126	Secure Coding	Input Validation	Identify all data sources and classify them into trusted and untrusted.	Validate all data from untrusted sources (e.g., Databases, file streams, etc).			√	√
11.1.1_127	Secure Coding	Input Validation	There should be a centralized input validation routine for the application.				√	√
11.1.1_128	Secure Coding	Input Validation	Specify proper character sets, for all sources of input.	Such as UTF-8			√	√
11.1.1_129	Secure Coding	Input Validation	Encode data to a common character set before validating (Canonicalize).				√	√
11.1.1_130	Secure Coding	Input Validation	All validation failures should result in input rejection.				√	√
11.1.1_131	Secure Coding	Input Validation	Determine if the system supports UTF-8 extended character sets and if so, validate after UTF-8 decoding is completed.				√	√
11.1.1_132	Secure Coding	Input Validation	Validate all client provided data before processing, including all parameters, URLs and HTTP header content (e.g. Cookie names and values). Be sure to include automated post backs from JavaScript, Flash or other embedded code.				√	√
11.1.1_133	Secure Coding	Input Validation	Validate for expected data types.				√	√
11.1.1_134	Secure Coding	Input Validation	Validate data range.				√	√
11.1.1_135	Secure Coding	Input Validation	Validate data length.				√	√
11.1.1_136	Secure Coding	Input Validation	Validate all input against a "white" list of allowed characters, whenever possible.				√	√
11.1.1_137	Secure Coding	Input Validation	If any potentially hazardous characters must be allowed as input, be sure that you implement additional controls like output encoding, secure task specific APIs and accounting for the utilization of that data throughout the application. Examples of common hazardous characters include: < > " ' % ( ) & + \ \' \"				√	√

11.1.1_138	Secure Coding	Input Validation	If your standard validation routine cannot address the following inputs, then they should be checked discretely	o Check for null bytes (%00) o Check for new line characters (%0d, %0a, \r, \n) o Check for “dot-dot-slash" (../ or ..\) path alterations characters. In cases where UTF-8 extended character set encoding is supported, address alternate representation like: %c0%ae%c0%ae/ (Utilize canonicalization to address double encoding or other forms of obfuscation attacks)			√	√
11.1.1_139	Secure Coding	Input Validation	A REST request or response body should match the intended content type in the header. Otherwise this could cause misinterpretation at the consumer/producer side and lead to code injection/execution.	Document all supported content types in your API			√	√
11.1.1_140	Secure Coding	Logical Access Control	Use only trusted system objects, e.g. server side session objects, for making access authorization decisions.			√	√	√
11.1.1_141	Secure Coding	Logical Access Control	Use a single site-wide component to check access authorization. This includes libraries that call external authorization services.			√	√	√
11.1.1_142	Secure Coding	Logical Access Control	Logical Access controls should fail securely			√	√	√
11.1.1_143	Secure Coding	Logical Access Control	Deny all access if the application cannot access its security configuration information.			√	√	√
11.1.1_144	Secure Coding	Logical Access Control	Segregate privileged logic from other application code.			√	√	√
11.1.1_145	Secure Coding	Logical Access Control	Restrict access to files or other resources, including those outside the application's direct control, to only authorized users.			√	√	√
11.1.1_146	Secure Coding	Logical Access Control	Restrict access to protected functions to only authorized. users			√	√	√
11.1.1_147	Secure Coding	Logical Access Control	Restrict access to API Services to only authorized users.	The API Standard and baseline must be used and adhered to when coding/using APIs.		√	√	√
11.1.1_148	Secure Coding	Logical Access Control	Restrict direct object references to only authorized users.			√	√	√
11.1.1_149	Secure Coding	Logical Access Control	Restrict access to services to only authorized users.			√	√	√
11.1.1_150	Secure Coding	Logical Access Control	Restrict access to application data to only authorized users.			√	√	√
11.1.1_151	Secure Coding	Logical Access Control	Restrict access to user and data attributes and policy information used by access controls.			√	√	√
11.1.1_152	Secure Coding	Logical Access Control	Restrict access security-relevant configuration information to only authorized users.			√	√	√
11.1.1_153	Secure Coding	Logical Access Control	Server side implementation and presentation layer representations of logical access control rules must match.			√	√	√
11.1.1_154	Secure Coding	Logical Access Control	If state data must be stored on the client, use encryption and integrity checking on the server side to catch state tampering.			√	√	√
11.1.1_155	Secure Coding	Logical Access Control	Enforce application logic flows to comply with business rules.			√	√	√

11.1.1_156	Secure Coding	Logical Access Control	Limit the number of transactions a single user or device can perform in a given period of time. The transactions/time should be above the actual business requirement, but low enough to deter automated attacks	To be determined per application based on "normal transactions/time behaviour baseline" and an upper treshold to prevent unusual high transaction rates/volumes.		√	√	√
11.1.1_157	Secure Coding	Logical Access Control	Use the "referer" header as a supplemental check only, it should never be the sole authorization check, as it is can be spoofed			√	√	√
11.1.1_158	Secure Coding	Logical Access Control	Implement account auditing and enforce the disabling of unused accounts (e.g., After no more than 30 days from the expiration of an account's password.) as defined in the Identity & Account baseline.			√	√	√
11.1.1_159	Secure Coding	Logical Access Control	The application must support disabling of accounts and terminating sessions when authorization ceases (e.g., Changes to role, employment status, business process, etc.).			√	√	√
11.1.1_160	Secure Coding	Logical Access Control	Service accounts or accounts supporting connections to or from external systems should have the least privilege possible.			√	√	√
11.1.1_161	Secure Coding	Logical Access Control	Create a specific application Access Control Policy document containing the application's business rules, data types and access authorization criteria and/or processes so that access can be properly provisioned and controlled. This includes identifying access requirements for both the data and system resources.	The Logical Access Control Policy, Standard and associated Baselines of the Security Management Framework (SMF) must be adhered to and used as mandatory input for when setting up an application Access Control Policy document.		√	√	√
11.1.1_162	Secure Coding	Logical Access Control	No anonymous access (including anonymous connections) is allowed, unless the purpose is to share information publicly in line with the classification, as decided by the business owner.	When anonymous access is permitted, access must be in line with the access control policy and logs must be created and stored for audit purposes that only public classified information and assets are shared.		√	√	√
11.1.1_163	Secure Coding	Memory Management	Utilize input and output control for un-trusted data			√	√	√
11.1.1_164	Secure Coding	Memory Management	Double check that the buffer is as large as specified			√	√	√
11.1.1_165	Secure Coding	Memory Management	When using functions that accept a number of bytes to copy, such as strncpy(), be aware that if the destination buffer size is equal to the source buffer size	It may not NULL-terminate the string		√	√	√
11.1.1_166	Secure Coding	Memory Management	Check buffer boundaries if calling the function in a loop and make sure there is no danger of writing past the allocated space			√	√	√
11.1.1_167	Secure Coding	Memory Management	Truncate all input strings to a reasonable length before passing them to the copy and concatenation functions.			√	√	√
11.1.1_168	Secure Coding	Memory Management	Specifically close resources, don't rely on garbage collection. (e.g., connection objects, file handles, etc.).			√	√	√
11.1.1_169	Secure Coding	Memory Management	Use non-executable stacks when available.			√	√	√
11.1.1_170	Secure Coding	Memory Management	Avoid the use of known vulnerable functions (e.g., printf, strcat, strcpy etc.).			√	√	√
11.1.1_171	Secure Coding	Memory Management	Properly free allocated memory upon the completion of functions and at all exit points.			√	√	√
11.1.1_172	Secure Coding	Output encoding	Conduct all encoding on a trusted system (e.g., The server).				√	√

11.1.1_173	Secure Coding	Output encoding	Utilize a standard, tested routine for each type of outbound encoding.				√	√
11.1.1_174	Secure Coding	Output encoding	Contextually output encode all data returned to the client that originated outside the application's trust boundary. HTML entity encoding is one example, but does not work in all cases.				√	√
11.1.1_175	Secure Coding	Output encoding	Encode all characters unless they are known to be safe for the intended interpreter.				√	√
11.1.1_176	Secure Coding	Output encoding	Contextually sanitize all output of un-trusted data to queries for SQL, XML, and LDAP.				√	√
11.1.1_177	Secure Coding	Output encoding	Command execution should not be allowed from the application.				√	√
11.1.1_178	Secure Coding	Output encoding	Sanitize all output of un-trusted data to operating system commands.				√	√
11.1.1_179	Secure Coding	Secure design	Use threat modeling for critical authentication, access control, business logic, and key flows			√	√	√
11.1.1_180	Secure Coding	Secure design	Integrate security language and controls into user stories			√	√	√
11.1.1_181	Secure Coding	Secure design	Integrate plausibility checks at each tier of your application (from frontend to backend)			√	√	√
11.1.1_182	Secure Coding	Secure design	Write unit and integration tests to validate that all critical flows are resistant to the threat model. Compile use cases and misue-cases for each tier of your application.			√	√	√
11.1.1_183	Secure Coding	Secure design	Segregate tier layers on the system and network layers depending on the exposure and protection needs.			√	√	√
11.1.1_184	Secure Coding	Secure design	Segregate tenants robustly by design throughout all tiers.			√	√	√
11.1.1_185	Secure Coding	Secure design	Limit resource consumption by user or service.			√	√	√
11.1.1_186	Secure Coding	Secure design	Implement a SDLC process for application design, development, deployment, and operation in accordance with security requirements defined by the organization.			√	√	√
11.1.1_187	Secure coding	Secure design	Philips Container/microservices baseline must be complied when microservices are running within a container or a groups containers.			√	√	√
11.1.1_188	Secure Coding	Security configuration	Implement automation for secure, standardized and compliant applications deployment.			√	√	√
11.1.1_189	Secure coding	Security configuration	Define and Implement automatic remediation process to remmediate application vulnerabilitites.			√	√	√
11.1.1_190	Secure Coding	Security configuration	Remove unused dependencies, unnecessary features, components, files, and documentation.			√	√	√
11.1.1_191	Secure Coding	Security configuration	Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies using tools like versions, OWASP Dependency Check, retire.js, etc. Continuously monitor sources like Common Vulnerability and Exposures (CVE) and National Vulnerability Database (NVD) for vulnerabilities in the components. Use software composition analysis tools to automate the process. Subscribe to email alerts for security vulnerabilities related to components you use.			√	√	√

11.1.1_192	Secure Coding	Security configuration	Only obtain components from official sources over secure links. Prefer signed packages to reduce the chance of including a modified, malicious component (See A08:2021-Software and Data Integrity Failures).			√	√	√
11.1.1_193	Secure Coding	Security configuration	Monitor for libraries and components that are unmaintained or do not create security patches for older versions. If patching is not possible, consider deploying a virtual patch to monitor, detect, or protect against the discovered issue.			√	√	√
11.1.1_194	Secure Coding	Security Logging	The Philips Logging Standard must be followed in addition to the specific controls for Logging and Error Handling & Logging mentioned within this standard.			√		
11.1.1_195	Secure coding	Security Logging	Monitor security related events within applications and the underlying infrastructure. Implement automation to send alerts to responsible stakeholders based on corresponding metrics.			√	√	√
11.1.1_196	Secure coding	Security Logging	Application owner must implement and document which logs, meta data and system events need to be logged.			√	√	√
11.1.1_197	Secure Coding	Security Logging	Ensure all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.			√	√	√
11.1.1_198	Secure Coding	Security Logging	Ensure that logs are generated in a format that log management solutions can easily consume.			√	√	√
11.1.1_199	Secure Coding	Security Logging	Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.			√	√	√
11.1.1_200	Secure Coding	Security Logging	Ensure high-value transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.			√	√	√
11.1.1_201	Secure Coding	Security Logging	Establish or adopt an incident response and recovery plan.			√	√	√
11.1.1_202	Secure Coding	Session Management	Use the server or framework's session management controls. The application should only recognize these session identifiers as valid.			√	√	
11.1.1_203	Secure Coding	Session Management	Session identifier creation must always be done on a trusted system (e.g., The server).			√	√	
11.1.1_204	Secure Coding	Session Management	Session management controls should use well vetted algorithms that ensure sufficiently random session identifiers.			√	√	
11.1.1_205	Secure Coding	Session Management	Set the domain and path for cookies containing authenticated session identifiers to an appropriately restricted value for the application.			√	√	
11.1.1_206	Secure Coding	Session Management	Logout functionality should fully terminate the associated session or connection.			√	√	
11.1.1_207	Secure Coding	Session Management	Logout functionality should be available from all pages protected by authorization.			√	√	
11.1.1_208	Secure Coding	Session Management	Establish a session inactivity timeout that is as short as possible, based on balancing risk and business functional requirements.	In most cases it should be no more than several hours. Define max 2 hours for timeout, or less depending on sensitivity of application or business requirements.		√	√	
11.1.1_209	Secure Coding	Session Management	If a session was established before login, close that session and establish a new session after a successful login.			√	√	

11.1.1_210	Secure Coding	Session Management	Generate a new session identifier on any re-authentication.			√	√	
11.1.1_211	Secure Coding	Session Management	Do not allow concurrent logins with the same user ID	Do not permit duplicate concurrent user sessions from different machines and verify that the application limits the number of active concurrent sessions.		√	√	
11.1.1_212	Secure Coding	Session Management	Protect server side session data from unauthorized access, by other users of the server, by implementing appropriate access controls on the server.			√	√	
11.1.1_213	Secure Coding	Session Management	Supplement standard session management for sensitive server-side operations, like account management, by utilizing per-session strong random tokens or parameters. This method can be used to prevent Cross Site Request Forgery attacks.			√	√	
11.1.1_214	Secure Coding	Session Management	Supplement standard session management for highly sensitive or critical operations by utilizing per-request, as opposed to per-session, strong random tokens or parameters.			√	√	
11.1.1_215	Secure Coding	Session Management	Enable the "secure" attribute for cookies transmitted over an TLS connection.	The TLS Connection must be 1.2 or higher		√	√	
11.1.1_216	Secure Coding	Session Management	Set cookies with the HttpOnly attribute, unless you specifically require client-side scripts within your application to read or set a cookie's value.			√	√	
11.1.1_217	Secure Coding	System Configuration	Ensure servers, frameworks and system components are running the latest approved version.			√	√	√
11.1.1_218	Secure Coding	System Configuration	Ensure servers, frameworks and system components have all patches issued for the version in use.			√	√	√
11.1.1_219	Secure Coding	System Configuration	Turn off directory listings.			√	√	√
11.1.1_220	Secure Coding	System Configuration	When exceptions occur, fail securely.			√	√	√
11.1.1_221	Secure Coding	System Configuration	Remove all unnecessary functionality and files.			√	√	√
11.1.1_222	Secure Coding	System Configuration	Remove test code or any functionality not intended for production, prior to deployment.			√	√	√
11.1.1_223	Secure Coding	System Configuration	Prevent disclosure of your directory structure in the robots.txt file by placing directories not intended for public indexing into an isolated parent directory. Then "Disallow" that entire parent directory in the robots.txt file rather than Disallowing each individual directory.			√	√	√
11.1.1_224	Secure Coding	System Configuration	Disable unnecessary HTTP methods, such as WebDAV extensions. If an extended HTTP method that supports file handling is required, utilize a well-vetted authentication mechanism.			√	√	√
11.1.1_225	Secure Coding	System Configuration	The security configuration store for the application should be able to be output in human readable form to support auditing.			√	√	√
11.1.1_226	Secure Coding	System Configuration	Implement an asset management system and register system components and software in it.	The asset management system must be set up according to the Asset Management Standard.		√	√	√

11.1.1_227	Secure Coding	System Configuration	Isolate development environments from the production network and provide access only to authorized development and test groups. Development environments are often configured less securely than production environments and attackers may use this difference to discover shared weaknesses or as an avenue for exploitation.			√	√	√
11.1.1_228	Secure Coding	System Configuration	Implement a software change control system to manage and record changes to the code both in development and production.			√	√	√
11.1.1_229	Secure Coding	Vulnerability testing Practices	Stake holder/Application owner must perform periodic penetration testing, when going to production and/or changes to the application. Testing must be performed by third parties.			√	√	√
11.1.1_230	Secure Coding	Website coding	When building, developing, coding a website, beside this secure coding baseline the Website secure Coding baseline must be adhered to.			√	√	√

Philips General Business Principles (GBP)
The Philips General Business Principles (GBP) apply to all employees of Philips and its controlled subsidiaries. The GBP are not all encompassing but formulate minimum standards of behavior. The underlying policies form an integral part of the GBP, including the information security policies and standards where this baseline is based on. If you are ever in doubt about a course of action, ask yourself the following question: Is it consistent with the content of the GBP? Is it ethical? Is it legal? Will it reflect well on Philips and myself? Would I want to be treated this way? Would I want to read about this in the newspaper? If you are uncertain, discuss the matter with your colleagues, seek guidance from either your manager or local GBP Compliance Officer, or go to:
https://share.philips.com/sites/Intranet_General_Business_Principles
References
Abbreviations and definitions used in this document can be found on the Group Security Portal:
https://share.philips.com/sites/STS020170831152504/gs/Lists/Security%20Terminology/AllItems.aspx
The Security Management Framework (SMF), consisting of all policies, standards, baselines and guidelines, can be found on the Group Security Portal:
https://share.philips.com/sites/STS020170831152504/gs/Library/Forms/Domain%20view.aspx
Other non-SMF documents that have contributed to this policy or that are referenced in this baseline are:
* Security Requirements; Philips IT – Reference page, Version 3.0:
https://share.philips.com/sites/CMSPUBLISHING020120920093047/Wiki/Pages/Requirements.aspx
* Secure Coding Guidelines; Philips IT – Reference page, Version 3.0.
https://share.philips.com/sites/CMSPUBLISHING020120920093047/Wiki/Pages/SecureCodingGuidelines.aspx
* Software Development at Philips (SW CoE)
* IT BIA Template
https://share-intra.philips.com/sites/STS20151001133003/SitePages/Home.aspx
* NIST Cloud Computing Reference Architecture (CCRA)
* Philips Brand Guidelines
https://www.ourbrand.philips.com/login/